

Introduction to Unit Testing (2/2)

pytest: Assertions, Fixtures, and Test Organization

SE 413/513 — Software Testing
Grand Valley State University

From Class 1 to Class 2

- Why `pytest` is generally a more widely accepted choice over `unittest`
- How a `pytest` test expresses an expected behavior with `assertions`.
- But real tests need **data, objects, files, databases, and other setup**.
- Today we learn how `pytest` manages that setup, controls its lifetime, and helps us avoid repetitive tests.

Key Idea

The central question for today is: **How do we keep a growing test suite clean, isolated, reusable, and maintainable?**

Learning Objectives

By the end of today, you should be able to:

- Create and use `pytest` fixtures as reusable test dependencies.
- Explain fixtures as a form of **dependency injection**.
- Use `yield`-based fixtures for reliable setup and teardown.
- Choose an appropriate fixture scope and explain the risk of shared mutable state.
- Use built-in fixtures such as `tmp_path` and `monkeypatch`.
- Use `@pytest.mark.parametrize` for data-driven testing.
- Share fixtures across a test suite using `conftest.py`.

Where We Are Going

- The problem fixtures solve
- Fixture creation and dependency injection
- Setup and teardown with `yield`
- Fixture scope and flaky tests
- Built-in fixtures
- Parametrized tests
- `conftest.py` and suite organization
- Practice and recap

Roadmap

- 1 Fixtures: The Basics
- 2 Setup and Teardown with `yield`
- 3 Fixture Scope
- 4 Built-in Fixtures
- 5 Parametrized Tests
- 6 `conftest.py` and Organizing a Suite
- 7 Recap
- 8 Practice: Fixtures, Parametrization, and Suite Design
- 9 Practice: Fixtures, Parametrization, and Suite Design

The Problem Fixtures Solve

Real tests rarely operate on bare literals. They need data, configured objects, temporary files, database connections — setup that is often shared across *many* tests.

pytest's answer: **fixtures** — functions decorated with `@pytest.fixture` that a test requests simply by naming it as a parameter.

Defining and Using a Fixture

```
import pytest

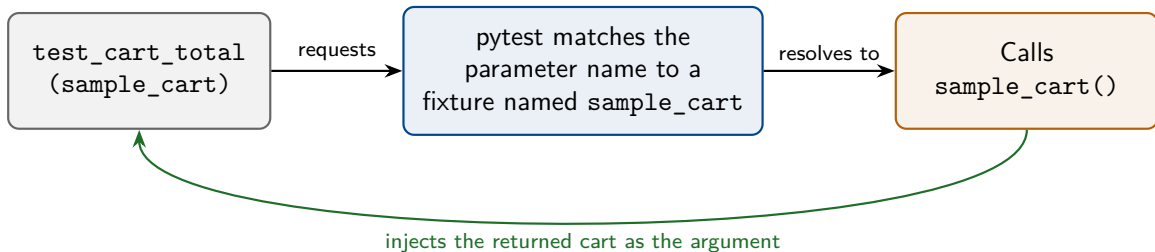
@pytest.fixture
def sample_cart():
    """Provide a shopping cart pre-loaded with two items."""
    return {"items": [
        {"name": "widget", "price": 9.99, "quantity": 3},
        {"name": "gadget", "price": 24.99, "quantity": 1},
    ]}

def cart_total(cart):
    return sum(i["price"] * i["quantity"] for i in cart["items"])

def test_cart_total(sample_cart):
    assert cart_total(sample_cart) == pytest.approx(54.96)
```

pytest sees the parameter `sample_cart`, recognizes it as a fixture name, calls that function, and passes its return value into the test.

Fixtures as Dependency Injection



Key Idea

The test *declares what it needs* as a parameter; pytest *supplies it*. Because a fixture is just a function, many tests can request the same one, and each test gets it prepared fresh by default — no shared state leaking between tests.

Fixtures Can Depend on Other Fixtures

Fixtures can themselves request other fixtures as parameters, building complex setups from small, composable pieces:

```
@pytest.fixture
def empty_cart():
    return {"items": []}

@pytest.fixture
def cart_with_one_item(empty_cart):
    empty_cart["items"].append(
        {"name": "widget", "price": 9.99, "quantity": 1})
    return empty_cart

def test_single_item_total(cart_with_one_item):
    assert cart_total(cart_with_one_item) == pytest.approx(9.99)
```

Tip

Because each test gets its own fresh call to `empty_cart()`, one test's mutations can never leak into another test — this is what “prepared fresh by default” bought you on the previous slide.

Roadmap

- 1 Fixtures: The Basics
- 2 Setup and Teardown with `yield`**
- 3 Fixture Scope
- 4 Built-in Fixtures
- 5 Parametrized Tests
- 6 `conftest.py` and Organizing a Suite
- 7 Recap
- 8 Practice: Fixtures, Parametrization, and Suite Design
- 9 Practice: Fixtures, Parametrization, and Suite Design

return vs yield (Python in general)

Both statements send a value back out of a function.

`return` **exits** the function and hands back one value for good. `yield` **pauses** the function, hands back one value, and picks up right where it left off the next time it's asked for another.

return – runs once

```
def count_up():  
    nums = []  
    for i in range(3):  
        nums.append(i)  
    return nums
```

```
result = count_up()  
print(result)  
# [0, 1, 2]
```

yield – runs lazily

```
def count_up():  
    for i in range(3):  
        yield i
```

```
result = count_up()  
print(next(result))  
# 0  
print(next(result))  
# 1
```

A function with `yield` becomes a **generator**: calling it doesn't run the body – it just creates an object that produces values **one at a time, on demand**, instead of building the whole list in memory at once.

Some Things Need Cleanup

Some resources must be cleaned up after a test finishes: closing a file, disconnecting from a database, deleting a temporary directory.

A fixture using `yield` instead of `return` splits naturally into a **setup** half (before the `yield`) and a **teardown** half (after it).

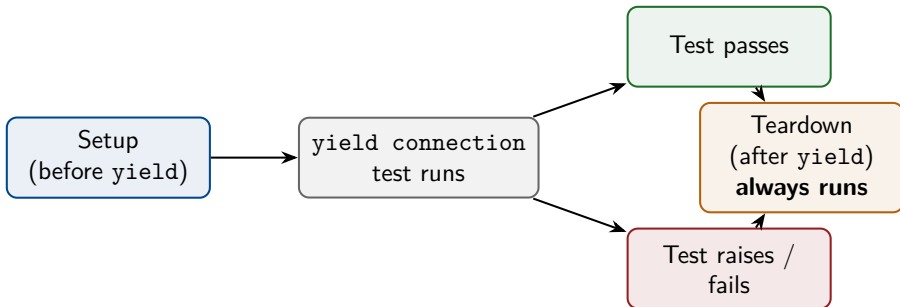
yield Fixtures: Setup + Teardown in One Function

```
import sqlite3, pytest

@pytest.fixture
def db_connection():
    connection = sqlite3.connect(":memory:")           # setup
    connection.execute(
        "CREATE TABLE users (id INTEGER PRIMARY KEY, name TEXT)")
    yield connection                                  # hand off
    connection.close()                                # teardown

def test_insert_and_query_user(db_connection):
    db_connection.execute("INSERT INTO users (name) VALUES ('Alice')")
    db_connection.commit()
    row = db_connection.execute("SELECT name FROM users").fetchone()
    assert row[0] == "Alice"
```

The `yield` Fixture Timeline



Key Idea

The code after `yield` runs **even if the test fails or raises** — which is exactly what makes `yield` fixtures the right tool for anything that must be cleaned up reliably.

Common Pitfall: `return` Has No Teardown

Common Pitfall

A fixture with `return` instead of `yield` has no teardown phase at all. If you find yourself wanting to write cleanup code “after the test,” that is the signal you need a `yield` fixture, not a plain one.

Roadmap

- 1 Fixtures: The Basics
- 2 Setup and Teardown with `yield`
- 3 Fixture Scope**
- 4 Built-in Fixtures
- 5 Parametrized Tests
- 6 `conftest.py` and Organizing a Suite
- 7 Recap
- 8 Practice: Fixtures, Parametrization, and Suite Design
- 9 Practice: Fixtures, Parametrization, and Suite Design

Why Scope Matters

By default, a fixture runs once *per test function* that requests it. For expensive setup — a real database, a large file, a server — re-running that setup for every test can make a suite unbearably slow.

The scope parameter controls how often a fixture is recreated.

The Four Scopes

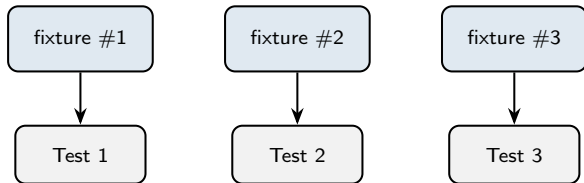
Scope	Fixture is (re-)created...
function (default)	Once per test function. Safest: no state leaks between tests.
class	Once per test class.
module	Once per test file.
session	Once for the entire test run, shared across every file.

```
@pytest.fixture(scope="module")
def db_connection():
    connection = sqlite3.connect(":memory:")
    connection.execute("CREATE TABLE users (id INTEGER PRIMARY KEY, name TEXT)")
    yield connection
    connection.close()
```

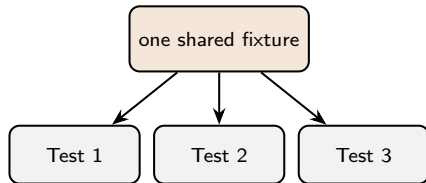
Every test in this file now shares one connection instead of paying setup/teardown cost per test.

Visualizing Scope Lifetime

`scope="function"`



`scope="module"`



Three separate instances vs. one shared instance for the same three tests. Sharing is faster — but now all three tests are touching the *same* object.

Common Pitfall: Wide Scope + Mutable State = Flaky Tests

Common Pitfall

Wider scope is a performance optimization, not a free upgrade. A session-scoped fixture holding *mutable* state (like a database where tests insert rows) can let one test's leftover data silently affect another test's result — a common source of **flaky tests**: tests that pass or fail unpredictably depending on execution order.

Default to function scope. Widen it only when you've measured a real performance problem, and make sure the fixture's state can't leak between tests when you do (e.g., roll back a transaction after each test).

Roadmap

- 1 Fixtures: The Basics
- 2 Setup and Teardown with `yield`
- 3 Fixture Scope
- 4 Built-in Fixtures**
- 5 Parametrized Tests
- 6 `conftest.py` and Organizing a Suite
- 7 Recap
- 8 Practice: Fixtures, Parametrization, and Suite Design
- 9 Practice: Fixtures, Parametrization, and Suite Design

tmp_path: A Safe Temporary Directory

Tests that read or write files should never touch real project files or a shared location. `tmp_path` provides a fresh, unique temporary directory (a `pathlib.Path`), auto-cleaned after each test.

```
def save_report(directory, filename, content):  
    path = directory / filename  
    path.write_text(content)  
    return path  
  
def test_save_report(tmp_path):  
    result_path = save_report(tmp_path, "report.txt", "Sales: $1,200")  
    assert result_path.exists()  
    assert result_path.read_text() == "Sales: $1,200"
```

No `@pytest.fixture` needed here — `tmp_path` ships with `pytest` itself.

monkeypatch: Safely Patching Things

monkeypatch temporarily changes attributes, dict entries, or environment variables for one test — and automatically undoes the change when the test ends, even on failure.

```
import os

def get_api_key():
    return os.environ["PAYMENT_API_KEY"]

def test_get_api_key(monkeypatch):
    monkeypatch.setenv("PAYMENT_API_KEY", "test-key-123")
    assert get_api_key() == "test-key-123"
```

Key Idea

Rule of thumb: reach for monkeypatch for simple attribute/env-var/dict substitutions; reach for MagicMock/patch() when you need a fully configurable fake that also records how it was called. (pytest also ships capsys, for capturing printed output — see the reading for a worked example.)

Roadmap

- 1 Fixtures: The Basics
- 2 Setup and Teardown with `yield`
- 3 Fixture Scope
- 4 Built-in Fixtures
- 5 Parametrized Tests**
- 6 `conftest.py` and Organizing a Suite
- 7 Recap
- 8 Practice: Fixtures, Parametrization, and Suite Design
- 9 Practice: Fixtures, Parametrization, and Suite Design

The Problem: One Test Per Case Doesn't Scale

Recall `calculate_late_fee()` and the negative-input fault from the *Testing Foundations* reading:

```
def calculate_late_fee(days_late, daily_rate=1.50, max_fee=25.00):  
    fee = days_late * daily_rate  
    if fee > max_fee:  
        return max_fee  
    return fee
```

Testing several inputs one at a time means a separate function per case:

```
def test_typical_fee():  
    assert calculate_late_fee(3) == 4.5  
  
def test_fee_is_capped():  
    assert calculate_late_fee(30) == 25.00
```

This does not scale as the number of cases grows.

@pytest.mark.parametrize: Data-Driven Tests

```
import pytest

@pytest.mark.parametrize("days_late, expected_fee", [
    (0, 0.0),          # no late fee
    (3, 4.5),          # typical case
    (30, 25.00),       # fee should be capped
    (100, 25.00),      # far past the cap, still capped
])

def test_calculate_late_fee(days_late, expected_fee):
    assert calculate_late_fee(days_late) == expected_fee
```

```
test_calculate_late_fee[0-0.0] PASSED
test_calculate_late_fee[3-4.5] PASSED
test_calculate_late_fee[30-25.0] PASSED
test_calculate_late_fee[100-25.0] PASSED
```

Four independent results, one per row — a single bad case fails on its own, without hiding that the other three still pass.

Documenting Known Bugs with xfail

The negative-day fault from *Testing Foundations* fits naturally into the same table:

```
@pytest.mark.parametrize("days_late, expected_fee", [
    (0, 0.0), (3, 4.5), (30, 25.00),
    pytest.param(-4, 0.0,
        marks=pytest.mark.xfail(reason="known bug: see Testing Foundations")),
])
def test_calculate_late_fee(days_late, expected_fee):
    assert calculate_late_fee(days_late) == expected_fee
```

xfail still *runs* the case, but reports XFAIL instead of FAIL — documenting a known defect in the suite itself without breaking the build, until it's fixed.

Parametrize as a Testing-Technique Vehicle

Tip

Parametrization is an excellent way to encode boundary-value and equivalence-partition test cases — techniques you will study formally very soon — directly into one readable test, instead of scattering nearly-identical test functions throughout a file.

Roadmap

- 1 Fixtures: The Basics
- 2 Setup and Teardown with `yield`
- 3 Fixture Scope
- 4 Built-in Fixtures
- 5 Parametrized Tests
- 6 `conftest.py` and Organizing a Suite**
- 7 Recap
- 8 Practice: Fixtures, Parametrization, and Suite Design
- 9 Practice: Fixtures, Parametrization, and Suite Design

Sharing Fixtures Across Files: `conftest.py`

Copy-pasting the same fixture into every test file becomes unmaintainable. `pytest` automatically discovers fixtures in a file named `conftest.py` — no import needed — and shares them with every test file in that directory *and its subdirectories*.

```
# tests/conftest.py
```

```
import pytest
```

```
@pytest.fixture
```

```
def sample_cart():
```

```
    return {"items": [{"name": "widget", "price": 9.99, "quantity": 3}]}
```

```
# tests/test_pricing.py -- no import of sample_cart needed
```

```
def test_cart_total(sample_cart):
```

```
    assert cart_total(sample_cart) == pytest.approx(29.97)
```

conftest.py Hierarchy

tests/ (conftest.py) -- visible to everything below

test_cart.py

tests/checkout/ (conftest.py)
-- visible only here

test_checkout.py

Key Idea

Fixtures defined closer to a test file (same directory, or a parent) are visible to it; fixtures in a *sibling* directory are not. This lets shared setup be scoped exactly as broadly, or narrowly, as it should be.

Beyond Today: Organizing a Growing Suite

A few conventions, not enforced by pytest but widely followed in practice:

- Mirror your source layout (`src/billing/invoices.py` → `tests/billing/test_invoices.py`).
- Name tests for *behavior*, not implementation (`test_fee_is_capped`, not `test_function_2`).
- Tag tests with **markers** (`@pytest.mark.slow`, `@pytest.mark.unit`) to run subsets selectively (`pytest -m "not slow"`); `@pytest.mark.skip/skipif` skip tests outright.

The reading covers directory conventions and marker registration in depth — treat this slide as a map of what's there, not a substitute for reading it.

Roadmap

- 1 Fixtures: The Basics
- 2 Setup and Teardown with `yield`
- 3 Fixture Scope
- 4 Built-in Fixtures
- 5 Parametrized Tests
- 6 `conftest.py` and Organizing a Suite
- 7 Recap**
- 8 Practice: Fixtures, Parametrization, and Suite Design
- 9 Practice: Fixtures, Parametrization, and Suite Design

A Practical Mental Model

When you see...	... think
<code>def test_x(some_name):</code>	pytest looks for a fixture named <code>some_name</code> and injects what it returns (or yields).
<code>yield</code> inside a fixture	Everything before is setup; everything after is teardown — and teardown runs even if the test fails.
<code>@pytest.mark.parametrize(...)</code>	One test function is really several independent test cases, each reported separately.
A fixture in <code>conftest.py</code>	Shared setup, visible to every test file in this directory and its subdirectories — no import required.

Key Takeaways

- **Prefer plain assertions.** Let pytest's assertion rewriting do the work of producing a useful failure message.
- **Fixtures are dependency injection for tests.** Declare what a test needs as a parameter; let pytest supply it.
- **Clean up reliably with `yield`.** Anything that must be closed, deleted, or disconnected belongs after a fixture's `yield`.
- **Default to narrow fixture scope.** Widen it deliberately, and only when state can't leak between tests.
- **Parametrize instead of duplicating.** A table of inputs and expected outputs beats a wall of near-identical test functions.
- **A test suite is a codebase.** Apply the same care to its structure, naming, and readability that you apply to production code.

One Last Question — and What's Next

Discussion Prompt

A teammate proposes making every fixture in the project session-scoped “for speed.” Using one idea from today and one from *Testing Foundations* (for example, the pesticide paradox, or context-dependent testing), argue for or against.

Next time: Equivalence Partitioning and Boundary-Value Analysis — the formal techniques for *choosing which inputs to test* in the first place, which is exactly the decision `@pytest.mark.parametrize` lets you encode.

Roadmap

- 1 Fixtures: The Basics
- 2 Setup and Teardown with `yield`
- 3 Fixture Scope
- 4 Built-in Fixtures
- 5 Parametrized Tests
- 6 `conftest.py` and Organizing a Suite
- 7 Recap
- 8 Practice: Fixtures, Parametrization, and Suite Design**
- 9 Practice: Fixtures, Parametrization, and Suite Design

Exercise: Create a Fixture

The same shopping cart data is needed by several tests. Turn the setup into a reusable pytest fixture and make the test request it as a parameter.

```
def cart_total(cart):  
    return sum(i["price"] * i["quantity"] for i in cart["items"])  
  
def test_cart_total():  
    cart = {"items": [  
        {"name": "widget", "price": 10, "quantity": 2}  
    ]}  
    assert cart_total(cart) == 20  
  
# Refactor the setup into a fixture.
```

Try it yourself before discussing with a partner.

Exercise: Add Reliable Cleanup

Suppose a test creates a resource that must always be closed. Complete the fixture so setup happens before the test and cleanup happens afterward, even when the test fails.

```
@pytest.fixture
def resource():
    connection = create_connection()
    # setup is complete

    # What belongs here?

    # What cleanup belongs here?
```

Try it yourself before discussing with a partner.

Exercise: Choose the Fixture Scope

Choose the most appropriate fixture scope for each situation and justify your choice.

- A small mutable shopping cart that each test modifies.
- A large read-only dataset loaded once for an entire test run.
- A temporary database connection that tests must not share.
- An expensive service setup shared by tests in one module, with careful state isolation.

Try it yourself before discussing with a partner.

Exercise: Parametrize the Cases

Replace the repetitive tests with one data-driven test using `@pytest.mark.parametrize`. Include at least three cases, including a boundary case.

```
def is_valid_age(age):  
    return 18 <= age <= 65  
  
# Replace these with one parametrized test:  
def test_valid_age_18():  
    assert is_valid_age(18) is True  
  
def test_valid_age_30():  
    assert is_valid_age(30) is True  
  
def test_invalid_age_17():  
    assert is_valid_age(17) is False
```

Try it yourself before discussing with a partner.

Exercise: `conftest.py` or Test File?

You have the same fixture used by five test files in `tests/`. Where should you put it? What if a fixture is needed only by tests under `tests/checkout/`?

Try it yourself before discussing with a partner.

Roadmap

- 1 Fixtures: The Basics
- 2 Setup and Teardown with `yield`
- 3 Fixture Scope
- 4 Built-in Fixtures
- 5 Parametrized Tests
- 6 `conftest.py` and Organizing a Suite
- 7 Recap
- 8 Practice: Fixtures, Parametrization, and Suite Design
- 9 Practice: Fixtures, Parametrization, and Suite Design

Exercise: Create a Fixture

The same shopping cart data is needed by several tests. Turn the setup into a reusable pytest fixture and make the test request it as a parameter.

```
def cart_total(cart):  
    return sum(i["price"] * i["quantity"] for i in cart["items"])  
  
def test_cart_total():  
    cart = {"items": [  
        {"name": "widget", "price": 10, "quantity": 2}  
    ]}  
    assert cart_total(cart) == 20  
  
# Refactor the setup into a fixture.
```

Try it yourself before discussing with a partner.

Exercise: Add Reliable Cleanup

Suppose a test creates a resource that must always be closed. Complete the fixture so setup happens before the test and cleanup happens afterward, even when the test fails.

```
@pytest.fixture
def resource():
    connection = create_connection()
    # setup is complete

    # What belongs here?

    # What cleanup belongs here?
```

Try it yourself before discussing with a partner.

Exercise: Choose the Fixture Scope

Choose the most appropriate fixture scope for each situation and justify your choice.

- A small mutable shopping cart that each test modifies.
- A large read-only dataset loaded once for an entire test run.
- A temporary database connection that tests must not share.
- An expensive service setup shared by tests in one module, with careful state isolation.

Try it yourself before discussing with a partner.

Exercise: Parametrize the Cases

Replace the repetitive tests with one data-driven test using `@pytest.mark.parametrize`. Include at least three cases, including a boundary case.

```
def is_valid_age(age):  
    return 18 <= age <= 65  
  
# Replace these with one parametrized test:  
def test_valid_age_18():  
    assert is_valid_age(18) is True  
  
def test_valid_age_30():  
    assert is_valid_age(30) is True  
  
def test_invalid_age_17():  
    assert is_valid_age(17) is False
```

Try it yourself before discussing with a partner.

Exercise: `conftest.py` or Test File?

You have the same fixture used by five test files in `tests/`. Where should you put it? What if a fixture is needed only by tests under `tests/checkout/`?

Try it yourself before discussing with a partner.

Exercise: Test a Function with monkeypatch

The function below reads the current username from the environment. Write a test that does not depend on the student's actual environment.

```
import os

def get_username():
    return os.environ.get("USERNAME", "guest")

def test_get_username():
    # Use monkeypatch to make the test deterministic.
    # What should you change and assert?
    pass
```

Try it yourself before discussing with a partner.

Exercise: Combine a Fixture and Parametrization

Use a fixture for the shared test data and `@pytest.mark.parametrize` for the different discount rates.

```
def discounted_price(price, rate):  
    return price * (1 - rate)  
  
# Use a fixture for price = 100 and  
# parametrize at least three discount rates.  
  
def test_discount():  
    # Write the test here.  
    pass
```

Try it yourself before discussing with a partner.

Questions?

Thank you.

SE 413/513 — Software Testing
Grand Valley State University